

Project Fundamentals — From Training to Authentication

A single, end-to-end explanation of the system as it stands today: how the models were trained on Kaggle, which datasets each modality used, what fine-tuning was done, how a biometric becomes a protected template, how similarity and fusion work, how the backend is organised, and how a face is registered. Every number below is taken from the repository (config files, `evaluation/results/*.csv`, `*_threshold.json`, training notebooks) unless it is explicitly marked as *not measured* or *not established*.

Companion documents with more depth on single topics: `DATASETS.md`, `VOICE_MODEL.md`, `COMPUTER_VISION.md`, `TEMPLATE_PROTECTION.md`, `MULTI_TEMPLATE_ARCHITECTURE.md`, `BACKEND_API.md`, `AUTHENTICATION_RELIABILITY_REPORT.md`.

1. What the system is

A privacy-preserving multimodal authentication system for critical-infrastructure access. A person is recognised by **face**, **fingerprint** and **voice** (iris code exists but has no trained model — see §3.4). The central design rule:

The raw image / audio and the raw embedding are never the stored credential. What is stored — and compared — is a **cancelable protected template**: a 256-bit string derived from the embedding under a secret key. If it leaks, the key version is rotated and the user gets a completely different template from the same face, without re-capturing anything.

End-to-end pipeline:

```
capture (image / audio)
-> modality preprocessing (classical CV / signal processing)      preprocessing/
-> embedding network (deep learning, fine-tuned on Kaggle)      models/
-> BioHash cancelable transform, keyed by HKDF-SHA256           template_protection/
-> 256-bit protected template (stored) / candidate template (compared)
-> Hamming similarity per modality vs. that modality's threshold backend/services/
-> fusion of the submitted modalities                            fusion/
-> ACCESS_GRANTED / ACCESS_DENIED / ENROLLMENT_REQUIRED        backend/api/fusion.py
```

Repository map: `preprocessing/` (per-modality CV), `models/` (network definitions, training code, saved checkpoints), `embeddings/` (pipelines tying preprocessing to models, face centroid), `template_protection/` (HKDF, BioHash, matcher), `fusion/` (score fusion, policies), `backend/` (FastAPI service, SQLite/SQLAlchemy), `frontend/` (React + Vite), `evaluation/` (metrics, experiments, calibration), `kaggle_kernels/` and `notebooks/` (training), `tests/` (452 tests), `config/buildings.json`.

2. How the Kaggle training is done

2.1 Why Kaggle, and how a run works

Training is scripted, not manual. Each modality has a Kaggle Kernel under

`kaggle_kernels/<modality>_training/` consisting of a `kernel-metadata.json` (title, attached dataset, GPU flag, internet flag) and a notebook. `scripts/run_kaggle_kernels.py` automates the whole cycle with the Kaggle CLI:

1. **push** — `kaggle kernels push -p kaggle_kernels/<modality>_training` starts the run on Kaggle's free GPU infrastructure;
2. **poll** — `kaggle kernels status <user>/<slug>` until `complete`;
3. **pull** — `kaggle kernels output ...` downloads the run output;
4. **install** — the resulting `.pt` and `.h5` checkpoints are copied into `models/<modality>/saved/`.

Inside each notebook: clone the repo (the specific branch — see §2.4), attach or download the dataset, preprocess with the repo's own `preprocessing/` code, fine-tune, **save the checkpoint** (`models/common/checkpoint_io.py` writes the canonical PyTorch `.pt` plus an `.h5` interoperability export), evaluate (accuracy / FAR / FRR / EER / ROC-AUC via `evaluation/`), and run a visual test of genuine vs impostor pairs. `notebooks/01-04_*.ipynb` are the interactive Colab equivalents of the same logic.

2.2 Shared training ingredients

- **Loss — ArcFace** (`models/common/arcface.py::ArcMarginProduct`, additive angular margin softmax; Deng et al., CVPR 2019). Every modality reuses this one implementation, so the recipes are comparable. ArcFace trains the embedding so that same-identity vectors cluster tightly by *angle*, which is exactly what cosine similarity later measures.
- **Embeddings are L2-normalised**, so cosine similarity = dot product.
- **Pretrained starting point, partial unfreezing**. Only the top layers are updated; earlier layers keep pretrained features. This is why a few thousand images are enough and why over-fitting is controlled by *how much* is unfrozen.
- **Device guard**. `detect_device()` runs a real tensor op on CUDA at start-up and falls back to CPU if the assigned GPU is unsupported (Kaggle sometimes assigns a Tesla P100, compute capability 6.0, which the preinstalled PyTorch build has no kernels for). The face, voice and first fingerprint runs actually trained on CPU because of this.

2.3 Per-modality training summary

	Face	Fingerprint	Voice	Iris
Backbone	InceptionResnetV1 (facenet-pytorch), pretrained on VGGFace2	ResNet50, pretrained on ImageNet (IMAGENET1K_V2)	ECAPA-TDNN (SpeechBrain module), trained with the repo's own pipeline	ResNet18 (ImageNet) + projection head
Embedding size	512	512	192	256
Dataset	LFW	SOCOFing "Real"	VoxCeleb1 subset (Indian celebrities)	CASIA-Iris-Thousand mirror
Loss	ArcFace + CrossEntropy	ArcFace + label smoothing 0.1 + hard-negative hinge	ArcFace (m 0.5, s 30)	ArcFace
Trained checkpoint in repo	yes	yes	yes	no

2.4 Problems hit while running real training (and the fixes)

Wrong branch cloned (fixed: clone the specific branch); `pip install facenet-pytorch` silently replaced Kaggle's GPU-matched `torch` and downgraded `numpy` (fixed: `--no-deps`, only in the face notebook); unsupported P100 (fixed: CUDA smoke-test + CPU fallback); iris data-loader missing imports; iris evaluation shape bug (`(H,W)` strip vs the 3-channel contract). Details: `PROJECT_REPORT.md` §8.3.

3. Datasets and fine-tuning, per modality

No dataset is committed to the repo; raw biometrics exist only inside the training runtime (`DATASETS.md`, `PRIVACY_AND_SECURITY.md`).

3.1 Face — LFW, fine-tuned FaceNet

- **Dataset:** LFW (Labeled Faces in the Wild), fetched in-kernel with `sklearn.datasets.fetch_lfw_people(min_faces_per_person=20, funneled=True, color=True)` → **3,023 images, 62 identities**. Non-commercial research license.
- **Model:** `InceptionResnetV1(pretrained='vggface2')` — already trained for face verification on VGGFace2 (~8.6k identities).
- **Preprocessing before training:** MTCNN detect + landmark alignment → 160×160 crop; pixels normalised `(x-127.5)/128`.
- **Fine-tuning:** everything frozen **except** `block8`, `last_linear`, `last_bn`; ArcFace head over the 62 identities; Adam, lr 1e-4; batch 32; **10 epochs**.
- **Result (from the run):** epoch 10 — train acc 0.976, val acc 0.913; **EER 0.010, AUC 0.999** (`evaluation/results/face_metrics.csv`).
- **Read this honestly:** the 62 identities are both the fine-tuning classes and the evaluation identities, so this is not an open-set benchmark on unseen people. The strong numbers mostly reflect the

VGGFace2 pretraining; the fine-tune adapts the top layers. It is also **not** evidence about real-webcam, real-world captures (see §7).

3.2 Fingerprint — SOCOFing, ResNet50 (substituting DeepPrint)

- **Dataset:** SOCOFing (Sokoto Coventry Fingerprint Dataset), Kaggle `ruizgara/socofing`; the unaltered "Real" folder only — 6,000 images, 600 subjects (identity, hand and finger encoded in the file name, e.g. `1_M_Left_index_finger.BMP`).
- **Why not DeepPrint:** DeepPrint has no released weights or reference implementation; ResNet50 + the shared ArcFace head is a reproducible substitute (`PROJECT_REPORT.md` §7.1).
- **Preprocessing (classical CV, `preprocessing/fingerprint.py`):** grayscale → CLAHE (clip 3, 8×8) → ridge normalisation (target mean 100 / var 100) → 3×3 Gaussian blur → **Gabor filter bank** (8 orientations, max response) → min-max normalise → resize 224 → 3-channel → ImageNet mean/std.
- **Model:** ResNet50 with named layers + projection head `2048→1024→BatchNorm→ReLU→Dropout(0.3)→512`, L2-normalised.
- **First attempt (superseded):** last block only, per-image split inside each subject → val acc 0.000, AUC 0.579. Two flaws were found: the split leaked identities across train/val/test, and capacity/regularisation were poorly matched to 600 classes.
- **Upgraded recipe (`models/fingerprint/`, `FingerprintConfig`):**
- **subject-disjoint split** 70/15/15 (each subject in exactly one split, seed 42) — validation/test people are never seen in training, so evaluation is genuine/impostor **pair** verification, not classification;
- freeze `conv1`, `bn1`, `layer1`; fine-tune `layer2-4` + head;
- ArcFace margin 0.5, scale 64, **label smoothing 0.1**;
- **balanced P×K batches** (16 identities × 4 samples = 64) so every batch has positives and negatives;
- **hard-negative mining:** from epoch 10, hinge penalty (margin 0.3, weight 0.1) on the most similar impostor pairs;
- AdamW (lr 3e-4, wd 1e-4), grad-clip 1.0, mixed precision, **3-epoch warm-up + cosine** to 1e-6, up to 30 epochs;
- **early stopping on validation EER** (patience 8, not before epoch 10 — an earlier setting stopped a real run at epoch 6);
- training-only augmentation (affine $\pm 12^\circ$ / $\pm 5\%$ / 0.9–1.1×, brightness/contrast, noise, motion blur, light elastic/grid distortion, random crop) — **no flips**, since a mirrored ridge pattern is not a valid fingerprint.
- **Result on held-out subjects (`fingerprint_metrics.csv`):** 900 test samples, 4,050 genuine / 400,500 impostor pairs; **EER 30.8 %**, **AUC 0.764**. Better than the first attempt, still weak. **Fingerprint is the weakest modality**; ~10 impressions per subject and 600 classes are hard for this recipe.

3.3 Voice — VoxCeleb1 subset, ECAPA-TDNN

- **Dataset:** Kaggle `gaurav41/voxceleb1-audio-wav-files-for-india-celebrity` (DbCL-1.0), a **subset** of VoxCeleb1: **24 speakers, 4,857 utterances**, split per speaker 70/15/15 → 3,389 / 717 / 751. The full ~1,251-speaker corpus is too large to fetch unattended in one kernel session.
- **Preprocessing (`preprocessing/voice.py`, `numpy` + `scipy` only):** mono → resample 16 kHz → energy-threshold VAD → RMS loudness normalisation → 4 s segment (random crop when training, centre crop at inference) → **80-bin log-mel** (`n_fft` 400, hop 160, mean-normalised).

- **Model:** ECAPA-TDNN (Desplanques et al. 2020: TDNN + squeeze-excitation channel attention + multi-layer aggregation) → **192-d** L2-normalised speaker embedding. Because the repo's `BaseEmbedder` contract expects a 3-channel array, the 2-D spectrogram is stacked to pseudo-RGB and channel 0 is taken back out inside the model (lossless).
- **Training:** ArcFace (m 0.5, s 30), AdamW lr 1e-3 / wd 1e-4, cosine schedule, batch 64, mixed precision, **30 epochs, ~8 hours on CPU** (the assigned P100 was unsupported). Augmentation is configurable but was **disabled** in the committed run (`training_config.json` : `augmentation_enabled: false`).
- **Result:** on 751 held-out utterances — **EER 2.29 %, AUC 0.9967**; mean genuine cosine ≈ 0.895 , impostor ≈ 0.132 (`voice_metrics.csv`). Split is per-utterance within speaker, not the official VoxCeleb trial protocol, so the speakers overlap with training; treat it as a strong same-domain result, not as a claim about arbitrary microphones.

3.4 Iris — implemented, not trained

CASIA-Iris-Thousand (via an unofficial Kaggle mirror with unverified redistribution rights), ResNet18 + projection head. One run trained fully but a clean evaluated checkpoint was blocked by Kaggle's free GPU quota. `models/iris/saved/` is empty, so the iris service runs a deterministic mock embedding. **Iris is not offered in the web app** (the product uses face / fingerprint / voice).

4. Embeddings, similarity, and thresholds

There are **two different similarity scores** in the system; confusing them is the main source of misreading results.

4.1 Embedding cosine similarity (model-quality metric)

`cos(a, b) = a · b` for unit vectors, range $-1 \dots 1$. Used **offline** in `evaluation/` to compute genuine/impostor score distributions, FAR/FRR, EER and ROC-AUC (`evaluation/metrics.py`), and in voice enrollment quality grading (§6.3). It is never used in the live match decision, because the raw embedding is not stored.

4.2 Hamming similarity of protected templates (the live match score)

`template_protection/matcher.py::compare` : `similarity = (number of equal bits) / (total bits)`, range $0 \dots 1$; 1.0 = identical, ≈ 0.5 = unrelated. Both templates are 256 bits (`Settings.template_bits`). This is the score every threshold applies to.

Why it is not the same scale as cosine: the BioHash step keeps only the *sign* of 256 random projections, so template similarity is a compressed, non-linear function of embedding cosine. Measured mapping in this project:

Embedding cosine	Template (Hamming) similarity
≈ 0.95	≈ 0.90
≈ 0.75	≈ 0.80
≈ 0.50	≈ 0.72
≈ 0.30	≈ 0.64
≈ 0 (unrelated)	≈ 0.50

So a threshold of 0.90 means "embeddings nearly identical (cosine ~0.95)", far stricter than the 0.90 sounds.

4.3 Thresholds in force

Per-modality thresholds are read from `evaluation/results/<modality>_threshold.json` by `backend/threshold_loader.py` (cached per process — **restart the backend after editing**); if a file is absent the fallback is `Settings.match_threshold = 0.90`.

Modality	Threshold	Source
Face	0.80	operator-specified
Voice	0.80	operator-specified
Fingerprint	0.90	fallback (no file)
Iris	0.90	fallback (no file)

The 0.80 values are operator choices, **not** the output of a genuine/impostor calibration run, so no FAR/FRR/EER is claimed for them. Face was briefly set to 0.72 and **reset to 0.80**: lowering it would also make AI-generated faces more likely to pass (§7).

5. Template protection (cancelable biometrics)

`template_protection/`, documented in `TEMPLATE_PROTECTION.md`.

1. **Key derivation — HKDF-SHA256** (`hkdf_keys.py`). Inputs: server-side `MASTER_SECRET` and the context `application_id | user_id | modality | key_version`. The context is hashed to a fixed-length salt; three separate HKDF calls (different `info` strings) yield three independent 32-byte seeds: **projection**, **permutation**, **threshold**. Nothing is stored; keys are re-derived on demand.
2. **BioHash generation** (`biohash.py`, `transform.py`):
3. L2-normalise the embedding;
4. **project** with a key-seeded random orthonormal matrix (QR of a Gaussian matrix) → 256 values in $[-1, 1]$;
5. **quantise** each value to a bit against a key-seeded random threshold centred on 0 (jitter $\propto 0.5 \cdot \text{std}$ of the projection);
6. **permute** the bits with a key-seeded permutation. Output: 256 bits, stored packed in `protected_templates`.
7. **Match**: derive the same key, run the *live* embedding through the same transform, compare bits with Hamming similarity.

8. **Revocation / diversity:** change `key_version` (or the application id) and every seed changes → a statistically unrelated template from the same biometric (mean pairwise Hamming similarity 0.500 between sets, `template_set_diversity.csv`).

Stated limits (not overclaimed): many-to-one and lossy, but **not** a proven one-way function; if `MASTER_SECRET` and the template both leak, partial reconstruction is a known attack class. There is no formal security proof.

5.1 Template sets (T1–T4)

One credential = **one template per enrolled modality**, and a set is that credential at one key version. Each user gets a pool of **four sets** (`TEMPLATE_POOL_SIZE = 4`): T1 `ACTIVE`, T2–T4 `STANDBY`; revoked sets become `REVOKED` and key versions are never reused. Authentication uses only the `ACTIVE` set and a runtime check forbids mixing sets. Revoking makes the oldest standby set active for every modality at once **without re-capturing anything**; an empty pool returns 409 (re-enroll). Because there is no login, revoke / activate / generate first require a biometric match against the `ACTIVE` set (HTTP 403 otherwise).

6. Enrollment

`POST /enroll` (multipart). The backend embeds the capture once, generates the four template sets from that single embedding, stores only protected templates, and discards the embedding and the raw sample.

6.1 How a face is registered (one-time, five poses, centroid)

Implemented in `backend/services/face_enrollment.py`, `embeddings/centroid.py`, `preprocessing/face.py`; UI in `frontend/src/components/capture/GuidedFaceCapture.tsx`.

1. **Guided capture.** The UI walks the user through five steps with a circular face guide: **Front, Left, Right, Slight Up, Slight Down** (sent as `pose_front ... pose_down`). Each capture is checked immediately via `POST /enroll/face/check-pose` (stores nothing); a bad capture is retaken on the spot.
2. **Per pose, the backend runs:** MTCNN face detection → the existing landmark alignment to a 160×160 crop → one 512-d FaceNet embedding (the fine-tuned checkpoint).
3. **Rejection rule — only two reasons:** `NO_FACE` (MTCNN found nothing) or `BLURRY` (Laplacian variance of the aligned crop < 25; measured: normal webcam-quality crops 116–920, visibly blurred ≤ 34). Nothing else about a capture is judged.
4. **Centroid.** The valid unit embeddings are averaged and the mean is re-normalised: `centroid = normalize(mean(e_1 ... e_k))`, with **at least 3 valid poses** required (fewer → 422, nothing stored).
5. **Templates from the centroid only.** T1–T4 are produced by the unchanged HKDF + BioHash path from the centroid; the temporary per-pose embeddings are discarded immediately. A single `image` still enrolls a face from one capture.
6. **Authentication stays single-shot:** one live image → one embedding → compared with the `ACTIVE` template. No head-turn prompts.

What is established and what is not: the design intent is that a centroid sits in the middle of the person's pose cloud. On the only offline data available (one portrait with rotations/shifts/exposure changes) centroid

and single-image templates behaved alike (mean template similarity 0.926 vs 0.929). Real head-turn captures across real people have **not** been evaluated.

6.2 Fingerprint

A PNG/JPG/JPEG upload → preprocessing (§3.2) → embedding → templates.

6.3 Voice (two recordings, graded consistency)

The user records the phrase twice in one request (`image` + `confirm_image`). The two ECAPA embeddings are compared by cosine:

Cosine	Grade	Outcome
≥ 0.85	Excellent	enrolled
0.75 – 0.84	Good	enrolled
0.60 – 0.74	Fair	409 LOW_QUALITY_WARNING unless <code>accept_low_quality=true</code>
< 0.60	Poor	422 ENROLLMENT_INCONSISTENT , nothing stored

6.4 Enrollment status

Derived from ACTIVE-set rows plus an `enrollment_events` table: `NOT_REGISTERED` , `REGISTERED` , `UPDATED` , `RETRY_REQUIRED` (voice only).

7. Authentication and fusion

7.1 Flow (`POST /authenticate/fusion` , `backend/services/authentication.py`)

1. **The user chooses the modalities** to present ("Select biometric factors for this authentication session"). Buildings (`config/buildings.json`) carry only id, name, clearance level and description — the loader rejects any `required_modalities` .
2. If a submitted modality is **not enrolled** → `409 ENROLLMENT_REQUIRED` : nothing is evaluated, and it is audited as its own state, not as a failure.
3. Otherwise, **exactly the submitted modalities** are evaluated: preprocess → embed → BioHash with the ACTIVE set's key → Hamming similarity vs the stored template → compared with that modality's threshold.
4. Fuse and decide → `ACCESS_GRANTED` or `ACCESS_DENIED` .

7.2 Fusion definition

- `fusion_similarity` = **weighted average of the per-modality Hamming similarities, equal weights**, renormalised over the modalities actually submitted (`fusion/score_fusion.py`); an absent modality is never treated as zero.
- `fusion_distance` = `1 - fusion_similarity` .
- `fusion_threshold` = mean of the per-modality thresholds used (e.g. face 0.80 + voice 0.80 → 0.80).
- **Decision policy** (`fusion/config.py` , `fusion/policy.py`):

- **ALL_REQUIRED (default):** every submitted modality must individually pass its own threshold. This fixed an audit finding in which a strong modality could average away another modality's failure.
- **AT_LEAST_TWO :** with exactly three modalities, two must pass.
- **WEIGHTED :** the fused average must reach the fusion threshold, but any modality below a 0.5 floor vetoes.

So under the default policy the fused score is informational and the gate is "all presented factors pass individually". The client receives only `fusion_similarity`, `fusion_distance`, `fusion_threshold`, `authentication_state`, `matched_modalities`, `active_template_set` and `key_version`. Per-modality scores appear in responses only when `DEBUG_SCORES=true`, and always in the audit table. The denied screen shows no similarity.

8. Backend

FastAPI + SQLAlchemy on SQLite (`DATABASE_PATH / DATABASE_URL`), started with `python -m uvicorn backend.main:app`. `MASTER_SECRET` is required.

Layers

- `backend/api/` — thin HTTP routers; `backend/services/` — logic; `backend/database/` — models, CRUD, migration (`scripts/migrate_to_multi_template.py`), audit; `backend/security_validation.py`, `backend/threshold_loader.py`, `backend/buildings.py`, `backend/states.py`.
- `backend/services/base_service.py::ModalityService` — one class parametrised by modality (face / iris / fingerprint / voice): `enroll` (embed → derive key → BioHash → store per set) and `authenticate` (embed → BioHash with ACTIVE key → Hamming → threshold). Models load lazily and cached (memory was tuned after a Render out-of-memory investigation: `mmap` + `assign` checkpoint loading, no pretrained download on the serving path).
- `backend/services/{enrollment,authentication,template_sets,recording_quality,face_enrollment}.py` — the flows described above.

Endpoints: `POST /enroll`, `POST /enroll/face/check-pose`; `POST /authenticate/fusion`, `POST /authenticate`, `POST /verify/{face|iris|fingerprint|voice}`; `GET /templates/{user_id}`, `POST /templates/{user_id}/activate/{version}`, `POST /templates/{user_id}/generate`, `POST /revoke-template`; `GET /user/{user_id}`, `GET /user/{user_id}/enrollment-status`, `DELETE /user/{user_id}`; `GET /buildings`, `GET /building/{id}`; `GET /audit/{user_id}`, `GET /audit/system`, `DELETE /audit/{user_id}`; `GET /metrics/{modality}`, `GET /system/health`.

Data: `users`; `protected_templates` (per user × modality × application × key version: packed 256-bit template, set status ACTIVE/STANDBY/REVOKED, template/key versions); `audit_logs` (per-modality similarities, thresholds used, fusion values, policy, submitted / enrolled / authenticated modalities, state, latency, template & key versions); `enrollment_events`. No raw image, audio or embedding is ever stored.

Security validation (`backend/security_validation.py`): runtime checks on the template pipeline, including the rule that templates of different sets are never compared with each other.

Frontend (React/Vite): landing, building checkpoint, Register (three independent enrollment cards: guided face, fingerprint upload, voice with the Recording Quality panel), Authenticate (factor selection then

capture), Result, Template Management (pool of sets with biometric authorization), Template Protection explainer, Analytics.

9. What was found while making genuine users work

- Genuine users were denied at the 0.90 fallback because 0.90 template similarity $\approx$ embedding cosine 0.95, which webcam/microphone captures rarely reach (`AUTHENTICATION_RELIABILITY_REPORT.md`). Face/voice moved to 0.80; a real-user's logged **face** attempts (15) scored **0.738–0.859, mean 0.784, only 5 $\geq$ 0.80**; **voice** $\approx$ 0.79–0.91; fingerprint 1.0 (the same file resubmitted).
 - **Why real face scores are ~0.78 rather than ~0.9 is not established.** A five-pose centroid pulling frontal matches down is one hypothesis; untested on real captures.
 - A lower face threshold was rejected by the operator: FaceNet-style embeddings cannot tell an AI-generated face from a real one.
-

10. Limitations (current state)

- **No liveness / anti-spoofing** — printed photos, screen replays and AI-generated faces are not detected; the threshold is a weak lever for that.
- Fingerprint is weak (EER 30.8 %, AUC 0.764 on held-out subjects); iris has no trained model.
- Face and voice metrics come from small, same-domain datasets (62 LFW identities; 24 voice speakers) and are not open-set or real-device benchmarks.
- The 0.80 thresholds are operator-set, not calibrated; no FAR/FRR is claimed for them.
- The template scheme has no formal security proof; security rests on `MASTER_SECRET` and on embeddings not leaking.
- No login layer; management actions are authorised by biometric match, which is only as strong as the weakest matched modality.
- Research/capstone system, not a hardened product; SQLite storage.

Suggested next steps (not started): compare frontal-only against five-pose enrollment on real captures to see whether the centroid lowers frontal similarity; add a liveness / presentation-attack check; calibrate per-modality thresholds from genuine/impostor data collected on real devices; retrain fingerprint on more data/impressions; finish the iris checkpoint if iris is wanted.